

FLUXOS DE AUTENTICAÇÃO, DADOS, GESTÃO DE CREDENCIAIS E CRIPTOGRAFIA

Projeto: Organiza.AI - Gestão Inteligente de Eventos em Grupo

Responsáveis: Diogo Mendes Baptista e Lucas do Nascimento Lima

Data: 21 de maio de 2026

Versão: 1.0

Classificação: Documentação Técnica

1 INTRODUÇÃO

Este documento detalha os mecanismos de autenticação, fluxos de dados, gestão de credenciais e implementação de criptografia no sistema Organiza.AI. A implementação segue os padrões RFC 7519 (JWT) e RFC 6238 (TOTP), com foco em proteção contra ataques comuns e conformidade com boas práticas de segurança [1](#) [2](#).

2 FLUXO DE AUTENTICAÇÃO DETALHADO

2.1 Fase 1: Registro de Usuário

O registro inicial estabelece as credenciais primárias do usuário:

Plain Text

- Usuário fornece dados:
 - Email (validado com regex)
 - Senha (mínimo 8 caracteres, com maiúsculas, minúsculas, números, símbolo)
 - Nome completo
 - Telefone (opcional)
- Validações no servidor:
 - Email único no banco de dados
 - Força da senha (OWASP recomendações)
 - Conformidade com LGPD (consentimento explícito)
 - Termos de Serviço aceitos
- Hashing de senha com BCrypt:
 - Algoritmo: BCrypt (NIST recomendado)
 - Work Factor (Strength): 12

- └→ Salt: Gerado automaticamente (16 bytes)
- └→ Tempo de processamento: ~100-200ms
- └→ Hash armazenado no banco (nunca a senha em texto plano)

4. Usuário criado:

- └→ Status: ATIVO
- └→ 2FA: DESATIVADO (pode ativar depois)
- └→ Papéis: ROLE_PARTICIPANTE (padrão)
- └→ Email de confirmação enviado

2.2 Fase 2: Login Inicial (Email + Senha)

O login valida as credenciais primárias:

Plain Text

1. Usuário fornece:

- └→ Email
- └→ Senha

2. Servidor processa:

- └→ Busca usuário por email no banco
 - └→ Se não encontrado: Erro "Credenciais inválidas"
- └→ Valida senha com BCrypt:
 - └→ BCrypt.checkpw(senha_fornecida, hash_armazenado)
 - └→ Se falso: Incrementa contador de tentativas falhas
 - └→ Se > 5 tentativas: Bloqueia por 15 minutos
- └→ Se válido: Procede para 2FA

3. Proteção contra força bruta:

- └→ Contador de tentativas falhas por email
- └→ Bloqueio exponencial: 1min, 5min, 15min
- └→ IP logging para detecção de padrões
- └→ Notificação ao usuário de tentativas falhas

2.3 Fase 3: Autenticação 2FA/TOTP

A autenticação de dois fatores adiciona camada de segurança física:

Plain Text

1. Ativação inicial de 2FA:

- └→ Servidor gera Secret TOTP (32 bytes, base32)
- └→ Servidor gera QR Code (contém Secret codificado)

- └→ Usuário escaneia com Google Authenticator
- └→ Dispositivo sincroniza com servidor (RFC 6238)

2. Sincronização de Tempo:

- └→ Servidor e dispositivo sincronizam via NTP
- └→ TOTP baseado em timestamp Unix
- └→ Período: 30 segundos
- └→ Dígitos: 6 (código de 6 dígitos)
- └→ Algoritmo: HMAC-SHA1

3. Validação de código TOTP no login:

- └→ Usuário fornece código de 6 dígitos
- └→ Servidor calcula TOTP esperado:
 - └→ `time_counter = floor(current_time / 30)`
 - └→ `TOTP = HMAC-SHA1(secret, time_counter) % 10^6`
- └→ Valida com tolerância de ± 1 período (60 segundos)
 - └→ Protege contra dessincronização de relógio
- └→ Se válido: Procede para geração de JWT

4. Backup Codes:

- └→ Servidor gera 10 códigos de backup (8 dígitos cada)
- └→ Usuário armazena em local seguro
- └→ Cada código pode ser usado uma única vez
- └→ Permite acesso se dispositivo for perdido

2.4 Fase 4: Geração de JWT

O JWT é gerado após autenticação bem-sucedida:

Plain Text

1. Payload do JWT:

```
{
  "sub": "usuario_id_12345",
  "email": "usuario@example.com",
  "roles": ["ROLE_PARTICIPANTE"],
  "iat": 1621234567,
  "exp": 1621234567 + (15 * 60), // 15 minutos
  "jti": "uuid_unico_token"
}
```

2. Assinatura:

- └→ Algoritmo: HMAC-SHA384
- └→ Secret: Chave secreta de 256 bits (armazenada em .env)
- └→ Formato: header.payload.signature

└→ Verificação: Servidor valida assinatura em cada requisição

3. Tempo de Expiração:

- └→ Access Token: 15 minutos (curto, reduz risco)
- └→ Refresh Token: 7 dias (permite renovação)
- └→ Logout: Token adicionado à Blacklist até expiração

4. Retorno ao cliente:

```
{  
  "access_token": "eyJhbGciOiJIUzI4NCIs...",  
  "token_type": "Bearer",  
  "expires_in": 900,  
  "refresh_token": "eyJhbGciOiJIUzI4NCIs..."  
}
```

2.5 Fase 5: Requisições Autenticadas

Cada requisição à API deve incluir o JWT:

Plain Text

1. Cliente envia requisição:

GET /eventos
Authorization: Bearer eyJhbGciOiJIUzI4NCIs...

2. Filtro de Segurança (SecurityFilter):

- └→ Extrai token do header Authorization
- └→ Valida formato: "Bearer <token>"
- └→ Valida assinatura com chave secreta
- └→ Valida expiração ($iat + exp > now$)
- └→ Consulta Blacklist para tokens revogados
- └→ Se válido: Extrai usuário e papéis do payload

3. Autorização (AuthorizationFilter):

- └→ Verifica papéis do usuário
- └→ Valida acesso ao recurso solicitado
 - └→ Ex: Apenas ROLE_ORGANIZADOR pode deletar evento
- └→ Se autorizado: Requisição processada

4. Processamento:

- └→ Controller recebe requisição autenticada
- └→ Service executa lógica de negócio
- └→ Repository acessa banco de dados
- └→ Resposta retorna ao cliente

2.6 Fase 6: Logout e Invalidação de Token

O logout revoga o token JWT:

Plain Text

1. Cliente solicita logout:
POST /logout
Authorization: Bearer eyJhbGciOiJIUzM4NCIs...
2. Servidor processa:
 - └→ Extrai token do header
 - └→ Extrai tempo de expiração (exp)
 - └→ Insere token na Blacklist:
INSERT INTO token_blacklist (token, expiracao)
VALUES ('eyJhbGciOiJIUzM4NCIs...', '2026-05-21 15:30:00')
 - └→ Retorna sucesso ao cliente
3. Limpeza de Blacklist:
 - └→ Job agendado (a cada 1 hora)
 - └→ Remove tokens expirados da Blacklist
 - └→ Reduz tamanho da tabela
4. Efeito:
 - └→ Token não pode mais ser usado
 - └→ Logout é respeitado instantaneamente
 - └→ Usuário deve fazer login novamente

3 GESTÃO DE CREDENCIAIS

3.1 Armazenamento de Credenciais

As credenciais são armazenadas de forma segura no banco de dados:

Plain Text

Tabela: usuarios

id (PK)		INT	
email (UNIQUE)		VARCHAR(255)	
senha_hash		VARCHAR(60)	// BCrypt sempre gera 60 caracteres
nome		VARCHAR(255)	
telefone		VARCHAR(20)	
2fa_secret		VARCHAR(32)	// Base32 encoded
2fa_ativado		BOOLEAN	

status		ENUM('ATIVO', 'INATIVO', 'BLOQUEADO')	
data_criacao		TIMESTAMP	
data_ultima_login		TIMESTAMP	
tentativas_falhas		INT	// Contador para força bruta
bloqueado_ate		TIMESTAMP	// Para bloqueio temporário

3.2 Configuração de Credenciais Sensíveis

Credenciais sensíveis são armazenadas em arquivo `.env` (nunca em código):

Plain Text

```
# .env (não versionado no Git)
SPRING_DATASOURCE_URL=jdbc:mysql://localhost:3306/organiza_ai
SPRING_DATASOURCE_USERNAME=root
SPRING_DATASOURCE_PASSWORD=senha_segura_123
JWT_SECRET=chave_secreta_256_bits_base64_encoded
JWT_EXPIRATION=900000 // 15 minutos em ms
TOTP_WINDOW=1 // Tolerância de ±1 período
```

3.3 Boas Práticas de Gestão

- **Nunca armazenar senhas em texto plano:** Sempre usar BCrypt com Work Factor ≥ 12
- **Chaves secretas em variáveis de ambiente:** Nunca em código ou repositório
- **Rotação de chaves:** Implementar rotação periódica de JWT_SECRET
- **Auditoria de acesso:** Registrar todos os logins e tentativas falhas
- **Proteção contra força bruta:** Bloqueio temporário após N tentativas
- **Notificações de segurança:** Alertar usuário sobre logins suspeitos

4 CRIPTOGRAFIA IMPLEMENTADA

4.1 Criptografia de Senhas (BCrypt)

O BCrypt é o algoritmo padrão para hashing de senhas ³:

Plain Text

Características:

- ↳ Algoritmo: Blowfish (adaptado)
- ↳ Work Factor: 12 (configurável)
- ↳ Salt: 16 bytes (gerado automaticamente)
- ↳ Saída: 60 caracteres (sempre)

↳ Tempo: ~100-200ms por hash (proposital)
↳ Resistência: Força bruta impraticável

Exemplo em Java:

```
BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(12);  
String senhaHash = encoder.encode("minha_senha_123");  
// Resultado: $2a$12$R9h/cIPz0gi.URNNX3kh20PST9/PgBkqquzi.Ss7KIUg02t0jWMUm
```

Validação:

```
boolean valido = encoder.matches("minha_senha_123", senhaHash);  
// Resultado: true
```

4.2 Criptografia de Tokens JWT

Os tokens JWT são assinados com HMAC-SHA384 4 :

Plain Text

Estrutura do JWT:

header.payload.signature

Header:

```
{  
  "alg": "HS384",  
  "typ": "JWT"  
}
```

Payload:

```
{  
  "sub": "usuario_id",  
  "email": "usuario@example.com",  
  "roles": ["ROLE_PARTICIPANTE"],  
  "iat": 1621234567,  
  "exp": 1621234567 + 900,  
  "jti": "uuid_unico"  
}
```

Signature:

```
HMAC-SHA384(  
  base64(header) + "." + base64(payload),  
  secret_key_256_bits  
)
```

4.3 Criptografia de Dados em Trânsito

Todos os dados em trânsito são protegidos com TLS/SSL 5 :

Plain Text

```
Configuração Spring Boot:
server.ssl.enabled=true
server.ssl.key-store=classpath:keystore.p12
server.ssl.key-store-password=senha_keystore
server.ssl.key-store-type=PKCS12
server.ssl.key-alias=tomcat
```

Resultado:

```
├→ Protocolo: HTTPS (TLS 1.3)
├→ Certificado: Auto-assinado (desenvolvimento) ou CA (produção)
├→ Criptografia: AES-256-GCM
└→ Autenticação: HMAC-SHA256
```

4.4 Criptografia de Dados em Repouso

Dados sensíveis no banco de dados podem ser criptografados:

Plain Text

Campos sensíveis:

```
├→ Telefone: Criptografia opcional
├→ Endereço: Criptografia opcional
└→ Dados de pagamento: Criptografia obrigatória (PCI-DSS)
```

Implementação:

@Entity

```
public class Usuario {
    @Encrypted // Anotação customizada
    private String telefone;

    // Getters/Setters com criptografia automática
}
```

5 FLUXO DE DADOS COMPLETO

5.1 Fluxo de Criação de Evento

Plain Text

```
1. Cliente envia requisição:
  POST /eventos
  Authorization: Bearer <jwt>
  Content-Type: application/json
```

```
{
  "nome": "Churrasco de Aniversário",
  "descricao": "Celebração dos 30 anos",
  "local": "Parque da Cidade",
  "data_prevista": "2026-06-15"
}
```

2. EventoController recebe:

- └→ Valida JWT (autenticação)
- └→ Valida autorização (usuário pode criar evento)
- └→ Valida dados (nome não vazio, data no futuro, etc)

3. EventoService processa:

- └→ Cria objeto Evento
- └→ Define organizador = usuário autenticado
- └→ Define status = PLANEJAMENTO
- └→ Chama repository para persistência

4. EventoRepository persiste:

- └→ INSERT INTO eventos (nome, descricao, organizador_id, status)
- └→ Retorna evento com ID gerado
- └→ Dados criptografados em repouso

5. Resposta ao cliente:

```
HTTP 201 Created
{
  "id": 12345,
  "nome": "Churrasco de Aniversário",
  "organizador_id": 67890,
  "status": "PLANEJAMENTO",
  "data_criacao": "2026-05-21T10:30:00Z"
}
```

5.2 Fluxo de Votação de Datas

Plain Text

1. Organizador propõe datas:

```
POST /eventos/{id}/sugestoes-data
{
  "datas": ["2026-06-15", "2026-06-22", "2026-06-29"]
}
```

2. Participantes votam:

```
POST /sugestoes-data/{id}/votos
{
  "data_escolhida": "2026-06-22"
}
```

```
}
```

3. Sistema contabiliza:

- └→ Cada participante pode votar uma vez
- └→ Sistema conta votos por data
- └→ Data com maior adesão é confirmada
- └→ Status do evento muda para CONFIRMADO

4. Resultado:

- └→ dataConfirmada = "2026-06-22"
- └→ status = "CONFIRMADO"
- └→ Notificação enviada a todos os participantes

6 CONFORMIDADE COM PADRÕES

O sistema implementa conformidade com padrões de segurança reconhecidos:

Padrão	Implementação	Referência
RFC 7519	JWT com HMAC-SHA384	4
RFC 6238	TOTP para 2FA	1
OWASP	Proteção contra Top 10	6
NIST	BCrypt para hashing	3
LGPD	Consentimento e direitos	7

7 CONCLUSÃO

O Organiza.AI implementa um fluxo de autenticação robusto com múltiplas camadas de segurança, gestão segura de credenciais e criptografia de dados em trânsito e repouso. A conformidade com padrões internacionais garante proteção contra ataques comuns e conformidade regulatória.

REFERÊNCIAS

- [1] M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). TOTP: Time-Based One-Time Password Algorithm - RFC 6238. IETF. Disponível em:
- [2] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) - RFC 7519. IETF. Disponível em:

[3] NIST. (2019). Recommendation for Password-Based Key Derivation - SP 800-132. National Institute of Standards and Technology. Disponível em:

[4] HMAC-SHA384. (2002). FIPS 198-1: The Keyed-Hash Message Authentication Code (HMAC). NIST. Disponível em:

[5] TLS 1.3. (2018). RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3. IETF. Disponível em:

[6] OWASP. (2023). OWASP Top 10 Web Application Security Risks. Disponível em:

[7] Brasil. (2018). Lei Geral de Proteção de Dados Pessoais (LGPD). Lei nº 13.709/2018. Disponível em:

Histórico de Revisões

Versão	Data	Descrição	Autor
1.0	21/05/2026	Elaboração inicial dos fluxos de autenticação, dados e criptografia.	Manus AI